

FULL STACK DEVELOPMENT

FASE 2 | AULA 05 -

BANCO DE DADOS COLUNARES

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	4
SAIBA MAIS	5
MERCADO, CASES E TENDÊNCIAS	11
O QUE VOCÊ VIU NESTA AULA?	12
REFERÊNCIAS	13

EMANDA

O QUE VEM POR AÍ?

Boas-vindas!

Prosseguindo em nossos estudos, nesta aula vamos falar de um tipo de banco de dados cada vez mais utilizado no mercado: os bancos de dados colunares!

Exploraremos os conceitos e características desses sistemas de armazenamento de dados que têm revolucionado a forma como lidamos com grandes volumes de informações. Os bancos de dados colunares são uma alternativa aos bancos de dados relacionais. Ao contrário dos modelos relacionais, os bancos de dados colunares armazenam os dados em colunas, o que traz benefícios significativos para aplicações que lidam com análise e consulta de grandes conjuntos de dados.

Durante esta aula, daremos destaque especial ao Apache Cassandra, um dos principais sistemas de gerenciamento de bancos de dados colunares disponíveis. Veremos como o Cassandra oferece escalabilidade horizontal, tolerância a falhas e uma arquitetura distribuída, que o torna ideal para aplicações que exigem alta disponibilidade e desempenho em ambientes distribuídos.

HANDS ON

No primeiro vídeo, exploraremos os principais benefícios de se optar por uma base de dados colunar e as diferenças de modelagem em relação ao modelo relacional. Para isso, vamos tomar como exemplo um novo problema em nossa já conhecida plataforma para gestão de ingressos.

Na segunda videoaula, falaremos sobre o Apache Cassandra, compreendendo sua arquitetura única que garante resiliência a falhas, escalabilidade e replicação dos dados. Além disso, daremos os primeiros passos para a criação de uma base de dados colunar.

Na sequência, vamos colocar em prática todos os conceitos aprendidos nos vídeos anteriores. Continuando na nossa plataforma de ingressos, vista na última aula, vamos praticar exemplos de operações básicas de manipulação e consulta dos dados usando o Cassandra! Vamos lá?

Instalação

Após a instalação, o Cassandra estará pronto para uso. Você poderá se conectar ao servidor usando uma ferramenta de linha de comando, como o `cqlsh`, ou através de drivers e bibliotecas disponíveis para a linguagem de programação de sua escolha. Certifique-se de consultar a documentação oficial do Apache Cassandra para obter instruções detalhadas de instalação e configuração específicas para o seu ambiente.

Iniciando o servidor

Após a inicialização bem-sucedida, o servidor do Cassandra estará pronto para aceitar conexões e executar operações de banco de dados. Você pode se conectar ao Cassandra utilizando uma ferramenta de linha de comando, como o `cqlsh`, ou através de drivers e bibliotecas disponíveis para a linguagem de programação de sua escolha. Lembre-se de que, para interromper o servidor do Cassandra, você pode pressionar `Ctrl+C` no terminal onde ele está sendo executado.

SAIBA MAIS

Introdução a bancos de dados colunares

Bancos de dados colunares são sistemas de gerenciamento de banco de dados (SGBDs) que organizam e armazenam os dados em formato de colunas, em vez de linhas, como nos bancos de dados relacionais tradicionais. Nesse tipo de banco de dados, os dados de cada coluna são armazenados juntos em uma estrutura otimizada, permitindo um acesso rápido e eficiente.

Em um banco de dados colunar, as colunas são tratadas como a unidade de armazenamento e consulta. Cada coluna contém valores de um único tipo de dados e as linhas são compostas por conjuntos de valores correspondentes em todas as colunas. Isso permite que os bancos de dados colunares sejam altamente eficientes em consultas analíticas e agregações, onde apenas algumas colunas específicas são necessárias para responder às consultas. As principais características e vantagens dos bancos de dados colunares incluem:

- Alta taxa de compressão de dados, devido à similaridade dos valores em uma coluna;
- Melhor desempenho em consultas analíticas e agregações, pois apenas as colunas relevantes são acessadas;
- Armazenamento eficiente de grandes volumes de dados;
- Maior flexibilidade para adicionar e remover colunas sem afetar o desempenho ou a estrutura dos dados existentes;
- Suporte a consultas complexas, incluindo filtros por colunas e projeções seletivas.

Exemplos de bancos de dados colunares populares incluem o Apache Cassandra, o Apache HBase e o Amazon Redshift. Eles são frequentemente utilizados em cenários de big data, análise de dados, armazenamento de séries temporais e processamento de dados em tempo real.

Os bancos de dados colunares surgiram na década de 1970, com o desenvolvimento do banco de dados relacional. No entanto, se tornaram mais proeminentes e populares nos últimos anos com o advento da big data e a

necessidade de lidar com volumes massivos de dados e consultas analíticas complexas.

Uma das implementações pioneiras dos bancos de dados colunares foi o Vertica, desenvolvido na década de 2000. Ele foi projetado especificamente para consultas analíticas de alto desempenho e se tornou uma referência nesse campo. Desde então, muitos outros sistemas de bancos de dados colunares foram desenvolvidos.

É importante mencionar que o surgimento e a evolução dos bancos de dados colunares estão intrinsecamente ligados ao avanço da tecnologia, do processamento paralelo e da capacidade de armazenamento. À medida que essas tecnologias se desenvolveram, os bancos de dados colunares se tornaram mais acessíveis e eficientes para manipular grandes volumes de dados, oferecendo desempenho otimizado para consultas analíticas.

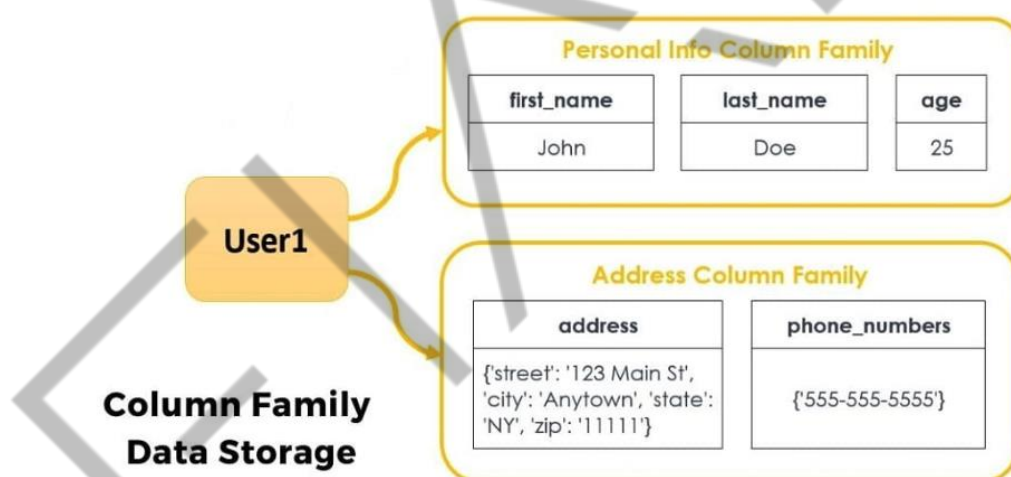


Figura 1 - Exemplos de famílias de colunas.

Fonte: Database Town (s.d.)

Aprofundando no Cassandra

Apache Cassandra é um sistema de banco de dados NoSQL de código aberto, altamente escalonável e descentralizado, projetado para lidar com grandes quantidades de dados em vários servidores comuns, ao mesmo tempo em que fornece alta disponibilidade e tolerância a falhas. Foi originalmente desenvolvido pelo Facebook e posteriormente de código aberto como parte da Apache Software Foundation.

Projetado para lidar com grandes volumes de dados distribuídos em várias máquinas, ele utiliza uma arquitetura descentralizada, baseada no modelo de replicação e particionamento, conhecido como "ring". Isso permite que os dados sejam distribuídos de maneira uniforme entre os nós do cluster, evitando pontos únicos de falha e garantindo a resiliência do sistema.

Principais características do Cassandra

Arquitetura Distribuída: possui uma arquitetura descentralizada ponto a ponto na qual os dados são dispersos entre vários nós de cluster. Isso permite um escalonamento simples, adicionando ou excluindo nós de acordo com os requisitos de dados.

Alta escalabilidade: ao dispersar automaticamente os dados pelo cluster, o Cassandra pode gerenciar enormes volumes de dados. Permite escalabilidade linear, o que aumenta a capacidade do cluster de lidar com mais dados e tráfego à medida que mais nós são adicionados.

Alta disponibilidade: garante que os dados estejam acessíveis mesmo quando um nó falha. Para garantir disponibilidade contínua, fornece replicação de dados em vários nós, permitindo failover automatizado e replicação de dados.

Nenhum ponto único de falha: não tem um único ponto de falha, pois é distribuído. Para garantir que os dados ainda estejam disponíveis em outros nós, mesmo se alguns falharem, ele replica os dados em vários nós.

Modelo de dados flexível: emprega um modelo de dados flexível baseado em família de colunas. Possui uma arquitetura livre de esquemas que permite adicionar e remover colunas sem alterar os dados já existentes.

Suporte para clustering e replicação: para maior velocidade e tolerância a falhas, Cassandra fornece soluções de clustering e replicação que dispersam dados em muitos data centers ou áreas geográficas.

Uma linguagem semelhante ao SQL, chamada Query Language (CQL), é usada para comunicação com bancos de dados, fornecendo uma sintaxe bem conhecida para interagir e acessar dados do Cassandra. Jogos online, redes sociais, Internet das Coisas (IoT), análises e muitos outros aplicativos que precisam de

escalabilidade, alta disponibilidade e tolerância a falhas frequentemente empregam o Apache Cassandra.



Figura 2 - Logotipo do Cassandra

Fonte: Cassandra (s.d.)

Modelagem de dados

A modelagem é crucial para o sucesso dos projetos no Cassandra. É necessário compreender o funcionamento do banco de dados antes de projetar o sistema. A eficiência nas operações de leitura e escrita depende de uma modelagem bem-feita, que envolve desnormalizar os dados. Geralmente é necessário criar uma tabela para cada tipo de consulta.

O Cassandra determina as partições por meio da chave de partição. Cada partição possui um coordenador, então quando uma aplicação precisa executar uma consulta e o nó que a recebe não tem os dados, ele redireciona a consulta para o coordenador da partição correspondente. A chave de partição deve ser especificada na cláusula "WHERE" de todas as consultas. Caso contrário, ele não conseguirá identificar o nó que possui os dados e espalhará a consulta por todo o cluster. Por exemplo, em um cenário com 2 datacenters, cada um com 2 racks (cada rack possui 5 máquinas), se a chave de partição não for informada, o Cassandra executará a consulta em todas as máquinas (totalizando 20 nós). Ter várias tabelas ajuda a mudar a chave de partição, garantindo que o campo relevante esteja na chave de partição sempre que necessário realizar uma consulta.

Como o Cassandra é altamente eficiente na persistência de dados, não há perda de desempenho com essa abordagem. A partir da versão 3.0, é possível criar uma visualização materializada, alterando a chave de partição no momento da criação da visualização. Com isso, a aplicação não precisa se preocupar em persistir os dados em N tabelas.

Transações e consistência

Embora o Cassandra não suporte transações, é possível realizar uma forma de persistência atômica. Isso envolve executar um bloco de N comandos de uma vez só, garantindo que os dados sejam persistidos apenas se todos os comandos forem executados com sucesso. No entanto, essa abordagem não melhora o desempenho, sendo apenas uma maneira de garantir a atomicidade na persistência de dados.

Devido ao tempo necessário para replicar os dados no cluster, consultas no Cassandra podem não retornar a versão mais recente dos registros, resultando em um problema de consistência. Para contornar esse problema, é possível configurar o nível de consistência (Consistency Level) tanto para operações de escrita quanto de leitura. Existem diferentes opções de níveis de consistência disponíveis, dependendo dos requisitos de cada aplicação.

Boas Práticas

- **Planeje sua modelagem de dados cuidadosamente:** a modelagem correta dos dados é fundamental. Considere os padrões de acesso aos dados, consultas necessárias e a distribuição dos dados nas partições. Evite modelagens excessivamente normalizadas e priorize a desnormalização para facilitar as consultas.
- **Entenda a consistência e a disponibilidade:** o Cassandra permite configurar a consistência e a disponibilidade dos dados com base nas necessidades do seu aplicativo. Compreenda os níveis de consistência e como eles afetam a escalabilidade, a latência e a disponibilidade dos dados.

- **Distribua os dados de forma equilibrada:** ele distribui os dados em várias partições. Certifique-se de distribuir os dados de forma equilibrada entre os nós do cluster para garantir uma divisão uniforme da carga de trabalho.
- **Otimize o esquema das tabelas:** utilize tipos de dados adequados para cada coluna e defina índices corretamente, considerando as consultas frequentes. Evite o uso excessivo de índices, pois isso pode impactar negativamente o desempenho.
- **Gerencie o armazenamento e a retenção de dados:** considere a capacidade de armazenamento do seu cluster e defina uma política adequada para a retenção de dados. Determine por quanto tempo os dados devem ser mantidos no Cassandra e implemente estratégias de exclusão e arquivamento de dados conforme necessário.
- **Monitore e ajuste o desempenho:** acompanhe regularmente o desempenho do cluster Cassandra, monitorando métricas importantes, como latência, taxa de transferência e utilização dos recursos. Ajuste as configurações de acordo com os requisitos do seu aplicativo e faça otimizações quando necessário.

Essas são apenas algumas das melhores práticas para trabalhar com o Cassandra. É sempre importante entender bem os conceitos e características do banco de dados e adaptar as práticas de acordo com as necessidades específicas do seu projeto.

MERCADO, CASES E TENDÊNCIAS

Cassandra na Netflix: um estudo de caso, no [blog Medium](#).

A fatia de mercado do Apache Cassandra entre as bases de dados NoSQL, no [site 6Sense](#).

Empresas que usam o Apache Cassandra, no [site enlyft](#).

EMANDA

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, exploramos os principais conceitos e características dos bancos de dados colunares e o Apache Cassandra.

Revisando o que aprendemos, vimos que os bancos de dados colunares são uma alternativa aos tradicionais bancos de dados relacionais, nos quais os dados são organizados em colunas, trazendo benefícios para aplicações que lidam com análise e consulta de grandes volumes de dados.

Em seguida, concentramos nossa atenção no Apache Cassandra, um poderoso sistema de gerenciamento de bancos de dados colunares. Discutimos sua arquitetura distribuída, escalabilidade horizontal e tolerância a falhas, o que o torna ideal para ambientes distribuídos.

Exploramos a modelagem de dados no Cassandra, destacando a estrutura de famílias de colunas e a flexibilidade do modelo colunar. Também discutimos os mecanismos de replicação e consistência, bem como as melhores práticas para projetar e otimizar consultas no Cassandra.

Agradecemos e até a próxima aula!

REFERÊNCIAS

BROWN, M. **Learning Apache Cassandra**. [s.l.]: Packt Publishing, 2015.

CARPENTER, J; HEWITT, E. **Cassandra: The Definitive Guide**. [s.l.]: O'Reilly Media, 2020.

PANIZ, D. **NoSQL: Como armazenar os dados de uma aplicação moderna**. [s.l.]: Casa do Código, 2016.

SADALAGE, P.J.; FOWLER, M. **NoSQL essencial: um guia conciso para o mundo emergente da persistência poliglota**. [s.l.]: Novatec Editora, 2019.

STRICKLAND, R. **Cassandra: High Availability**. [s.l.]: Packt Publishing, 2014.

PALAVRAS-CHAVE

Palavras-chave: Bancos de dados colunares, NoSQL, colunas, Cassandra.

EMSE

POS TECH